Day 72 — Line-by-Line Syntax Explanation

MD Mutasim Billah | 52-Week Data Science to ML/AI Roadmap | Week 11, Day 3 reference

Watch: Attention in transformers, step-by-step | Deep Learning Chapter 6 (3Blue1Brown) | Attention mechanism: Overview (Google Cloud Tech)

`day72_attention_heatmaps.py`, segment by segment, in the order `main()` uses things. Code boxes are cut from the real script by program (inline comments removed). Day 71's model and Day 70's probe are reused verbatim and not repeated. Purple **MATH** boxes come before code that implements a formula; the graph is the one the script saved. A glossary is at the end.

MATH BOX LEGEND

Symbols: W[m, j, i] = attention weight of sequence m, output step j, input position i · target(j) = the input position holding output j's digit · H = row entropy

Segment 0 — Setup

Settings

CODE

```
SEEDS = [0, 1, 2]
STEPS = 20000
SNAPSHOTS = {250, 500, 1000, 2000, 3000, 5000, 7500, 10000, 15000, 20000}
```

- `SNAPSHOTS = {250, 500, ...}` — a set: membership tests (step in SNAPSHOTS) are fast and order does not matter.
- `STEPS = 20000` — Day 71's budget, so the retrained models are Day 71's models.

Segment 1 — Models and snapshots

train_with_snapshots

CODE

```
def train_with_snapshots(task, hidden, seed, steps=STEPS, snap_at=SNAPSHOTS):
    net = AttnSeq2Seq(hidden=hidden, seed=seed)
    rng = np.random.RandomState(seed + 100)
    snaps = {0: copy.deepcopy(net.P)}
    for step in range(1, steps + 1):
        n = rng.randint(3, 21)
        Xs, Xd, Y, _ = make_batch(rng, 32, n, task)
        net.train_step(Xs, Xd, Y)
        if step in snap_at:
            snaps[step] = copy.deepcopy(net.P)
    return net, snaps
```

- `copy.deepcopy(net.P)` — a full copy of the weight dictionary; without it, later training would change the snapshot too.
- `snaps = {0: ...}` — the untrained weights are the first snapshot.
- `rng = np.random.RandomState(seed + 100)` — the same data stream as Day 71, so step for step the same training.

model and with_weights

CODE

```
def model(task, seed):
    if (task, seed) not in MODELS:
        MODELS[(task, seed)] = train_with_snapshots(task, 32, seed)
    return MODELS[(task, seed)]
def with_weights(P, hidden=32):
    net = AttnSeq2Seq(hidden=hidden)
    net.P = copy.deepcopy(P)
    return net
```

- `if (task, seed) not in MODELS:` — a tuple as a dictionary key: train each model once.
- `net.P = copy.deepcopy(P)` — put a snapshot's weights into a fresh model object, which then runs normally.

Segment 2 — Measuring the alignment

target_position and alignment_stats

MATH

$$offset = argmax_i\ W[j, i] - target(j); \quad H_j = -\Sigma_i\ W[j, i]\ log\ W[j, i]$$

CODE

```
def target_position(task, n):
    j = np.arange(n)
    return j if task == "copy" else n - 1 - j
def alignment_stats(net, task, n, m=300):
    Xs, _, Y, src = fixed_test(n, task, m)
    pred, W = net.generate(Xs, n + 1, return_weights=True)
    W = W[:, :n]
    target = target_position(task, n)
    argmax = W.argmax(axis=2)
    offset = argmax - target[None, :]
    on_target = W[:, np.arange(n), target]
    entropy = -(W * np.log(W + 1e-12)).sum(axis=2)
    correct = pred[:, :n] == Y[:, :n]
    return dict(offset=offset, on_target=on_target, entropy=entropy, correct=correct, W=W,
        src=src, pred=pred)
```

- `n - 1 - j` — REVERSE: output step j writes the digit from the mirrored position.
- `W[:, :n]` — drop the last output step (EOS), which has no target digit.
- `W.argmax(axis=2)` — for every sequence and output step, the input position with the largest weight.
- `W[:, np.arange(n), target]` — fancy indexing: in row j take column target[j], for every sequence at once.
- `-(W * np.log(W + 1e-12)).sum(axis=2)` — entropy per row; the tiny constant avoids log(0).

Printing one alignment

CODE

```
for j in range(12):
    print(f"out {pred[j]:>1}" + "".join(f"{int(round(100 * w)):>4}" if w >= 0.005 else "   ." for
        w in W[j]))
```

- `int(round(100 * w))` — a weight as a whole percentage.
- `... if w >= 0.005 else " ."` — weights that would round to 0 are shown as a dot, so the pattern stands out.

Next and previous states

CODE

```
nxt, prv = tgt + 1, tgt - 1
ok_n, ok_p = nxt < n, prv >= 0
rows.append([(st["offset"] == 0).mean(), st["on_target"].mean(),
             W[:, j[ok_n], nxt[ok_n]].mean(), W[:, j[ok_p], prv[ok_p]].mean(),
```

- `ok_n, ok_p = nxt < n, prv >= 0` — boolean masks: the last target has no next state, the first no previous one.
- `j[ok_n], nxt[ok_n]` — keep only the output steps where that neighbour exists.

Segment 3 — Experiments

The probe on single encoder states

CODE

```
Htr, Hte = net.encode(Xtr)[0], net.encode(Xte)[0]
p = 7
print(f"\n{'probe reads x_8 from':<22}" + "".join(f"{'h_' + str(q + 1):>8}" for q in [p - 1, p, p
    + 1, p + 2, p + 4]))
acc = [(fit_probe(Htr[:, q], src_tr[:, p])(Hte[:, q]) == src_te[:, p]).mean() for q in [p - 1, p,
    p + 1, p + 2, p + 4]]
```

- `net.encode(Xtr)[0]` — encode returns several things; [0] is every encoder state, shape (sequences, 20, 32).
- `Htr[:, q]` — one encoder state per sequence: position q.
- `for q in [p - 1, p, p + 1, p + 2, p + 4]` — before, at, and after the digit's position.

Following the snapshots

CODE

```
for step in sorted(SNAPSHOTS | {0}):
    vals = []
    for task in ["copy", "reverse"]:
        per = []
        for s in SEEDS:
            st = alignment_stats(with_weights(model(task, s)[1][step]), task, 12)
            per.append([(st["offset"] == 0).mean(), np.all(st["correct"], axis=1).mean()])
        curve[task].append(np.mean(per, axis=0)); vals += list(curve[task][-1])
    print(f"{step:>7}{vals[0]:>24.3f}{vals[1]:>8.3f}{vals[2]:>27.3f}{vals[3]:>8.3f}")
```

- `SNAPSHOTS | {0}` — set union: add step 0.
- `sorted(...)` — a set has no order; sorted() gives the steps in increasing order.
- `with_weights(model(task, s)[1][step])` — the snapshot's weights in a working model.
- `vals += list(...)` — extend a list with the two numbers of this task.

Right and wrong digits

CODE

```
right = np.concatenate([st["on_target"][st["correct"]] for st in sts])
wrong = np.concatenate([st["on_target"][~st["correct"]] for st in sts])
```

- `st["on_target"][st["correct"]]` — boolean indexing: the weights on target of the digits written right.
- `~st["correct"]` — ~ flips a boolean array: the digits written wrong.

Segment 4 — The figure

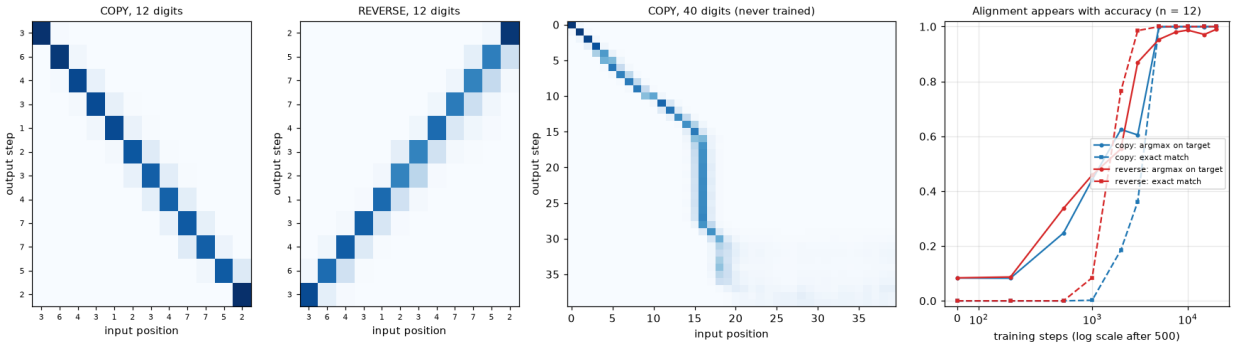
The heatmap figure

CODE

```
for k, (task, n, title) in enumerate([("copy", 12, "COPY, 12 digits"), ("reverse", 12, "REVERSE,
    12 digits"),
                                      ("copy", 40, "COPY, 40 digits (never trained)")]):
    ax = fig.add_subplot(gs[0, k])
    st = alignment_stats(model(task, 0)[0], task, n)
    ax.imshow(st["W"][0], cmap="Blues", vmin=0, vmax=1, aspect="auto")
    ax.set_xlabel("input position"); ax.set_ylabel("output step"); ax.set_title(title,
        fontsize=10)
    if n == 12:
        ax.set_xticks(range(n)); ax.set_xticklabels(st["src"][0], fontsize=7)
        ax.set_yticks(range(n)); ax.set_yticklabels(st["pred"][0][:n], fontsize=7)
```

- `fig.add_gridspec(1, 4, width_ratios=[...])` — four panels of different widths in one row.
- `ax.imshow(..., cmap="Blues", vmin=0, vmax=1)` — draw a matrix as colours; vmin and vmax fix the scale to 0..1 for every panel.
- `ax.set_xticklabels(st["src"][0])` — label the columns with the input digits themselves.

REAL GRAPH



Real graph from the script: COPY, REVERSE, a 40-digit COPY, and alignment during training.

Quick Syntax Glossary — new or reinforced constructs

- `copy.deepcopy(d)` — an independent copy of a dictionary of arrays.
- `A | B` — the union of two sets.
- `sorted(s)` — a sorted list from any collection.
- `W[:, np.arange(n), t]` — pick one column per row, for every sequence.
- `x[mask], x[~mask]` — the entries where a boolean mask is True, or False.
- `ax.imshow(M, cmap=..., vmin=0, vmax=1)` — a heatmap with a fixed colour scale.
- `fig.add_gridspec(1, 4, width_ratios=...)` — a row of panels with chosen widths.

End of Day 72 syntax reference. For the experiments and what they showed, see the Day 72 complete guide.